

PROYECTO 2

Minería de Grafos

Spotify Artist Feature Collaboration Network

Materia	Minería de Datos en Grafos
Profesor	Dr. Víctor Ortega
Metodología	KDG (Knowledge Discovery in Graphs)
Dataset	Spotify Artist Feature Collaboration Network (Kaggle)
Fecha entrega	Jueves 14 de mayo, 2025 — 13:00 hrs
Equipo	_____
Integrantes	Ivanna Alejandra Camerota Curiel Lambda Remiel Heredia Pérez Ángel Daniel Pérez Urbina

1. Descripción del Dataset y Pregunta de Investigación

1.1 Fuente de datos

El dataset Spotify Artist Feature Collaboration Network está disponible públicamente en Kaggle. Contiene información de artistas de Spotify, sus géneros musicales y las colaboraciones ("features") realizadas entre ellos.

Archivo	Descripción
nodes.csv	Un nodo por artista: spotify_id, name, followers, popularity, genres
edges.csv	Una arista por colaboración: id_0, id_1 (spotify_id de cada artista)

1.2 Modelo heterogéneo elegido

Se promovieron los géneros musicales a nodos propios (:Genre), generando un grafo heterogéneo con:

- Nodos :Artist — propiedades: spotify_id, name, followers, popularity
- Nodos :Genre — propiedades: name
- Relaciones :COLLABORATED_WITH — entre artistas
- Relaciones :HAS_GENRE — artista → género

1.3 Preguntas de investigación

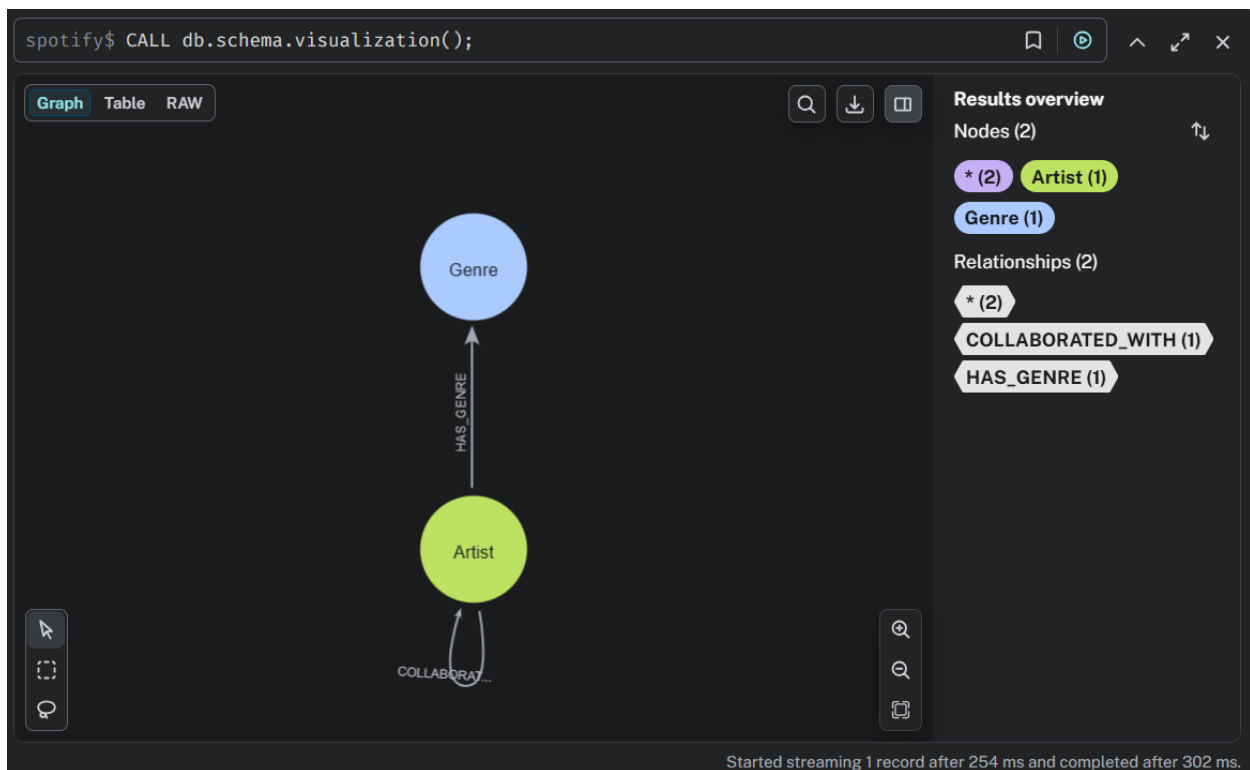
- P1. ¿Cuáles son los artistas más centrales en la red de colaboraciones?
- P2. ¿Existen comunidades de artistas que colaboran principalmente dentro de un mismo género?
- P3. ¿Cuál es el camino más corto entre dos artistas que nunca han colaborado directamente?
- P4. ¿Qué géneros agrupan más artistas y cómo están conectados entre sí?

2. Construcción del Grafo (ETL)

2.1 Script ETL (etl.py)

El script etl.py transforma los archivos originales del dataset en 4 CSVs listos para Neo4j:

- artists.csv → nodos :Artist
- genres.csv → nodos :Genre
- artist_genre.csv → relaciones :HAS_GENRE
- collaborations.csv → relaciones :COLLABORATED_WITH



2.2 Carga a Neo4j (LOAD CSV)

Los siguientes bloques Cypher crean los constraints, cargan nodos y relaciones. Ejecutarlos en orden desde Neo4j Browser.

Bloque 0 — Constraints e índices

Ejecutar primero. Garantiza unicidad y acelera los MERGE.

load_csv.cql — Bloque 0

```
CREATE CONSTRAINT artist_id IF NOT EXISTS
  FOR (a:Artist) REQUIRE a.spotify_id IS UNIQUE;

CREATE CONSTRAINT genre_name IF NOT EXISTS
  FOR (g:Genre) REQUIRE g.name IS UNIQUE;
```

Bloque 1 — Nodos :Artist

load_csv.cql — Bloque 1

```
LOAD CSV WITH HEADERS FROM 'file:///artists.csv' AS row
MERGE (a:Artist { spotify_id: row.spotify_id })
SET
  a.name          = row.name,
  a.followers     = toInteger(row.followers),
  a.popularity    = toInteger(row.popularity);
```

Bloque 2 — Nodos :Genre

load_csv.cql — Bloque 2

```
LOAD CSV WITH HEADERS FROM 'file:///genres.csv' AS row
MERGE (:Genre { name: row.name });
```

Bloque 3 — Relaciones :HAS_GENRE

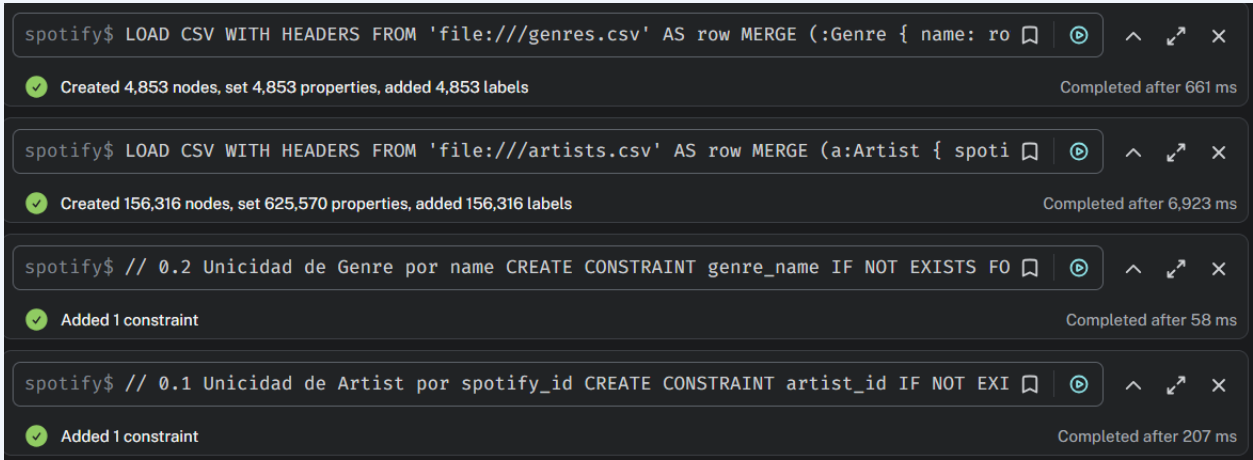
load_csv.cql — Bloque 3

```
:auto
LOAD CSV WITH HEADERS FROM 'file:///artist_genre.csv' AS row
CALL {
  WITH row
  MATCH (a:Artist { spotify_id: row.spotify_id })
  MATCH (g:Genre { name: row.genre })
  MERGE (a)-[:HAS_GENRE]->(g)
} IN TRANSACTIONS OF 500 ROWS;
```

Bloque 4 — Relaciones :COLLABORATED_WITH

load_csv.cql — Bloque 4

```
:auto
LOAD CSV WITH HEADERS FROM 'file:///collaborations.csv' AS row
CALL {
  WITH row
  MATCH (a:Artist { spotify_id: row.id_0 })
  MATCH (b:Artist { spotify_id: row.id_1 })
  MERGE (a)-[:COLLABORATED_WITH]->(b)
} IN TRANSACTIONS OF 500 ROWS;
```



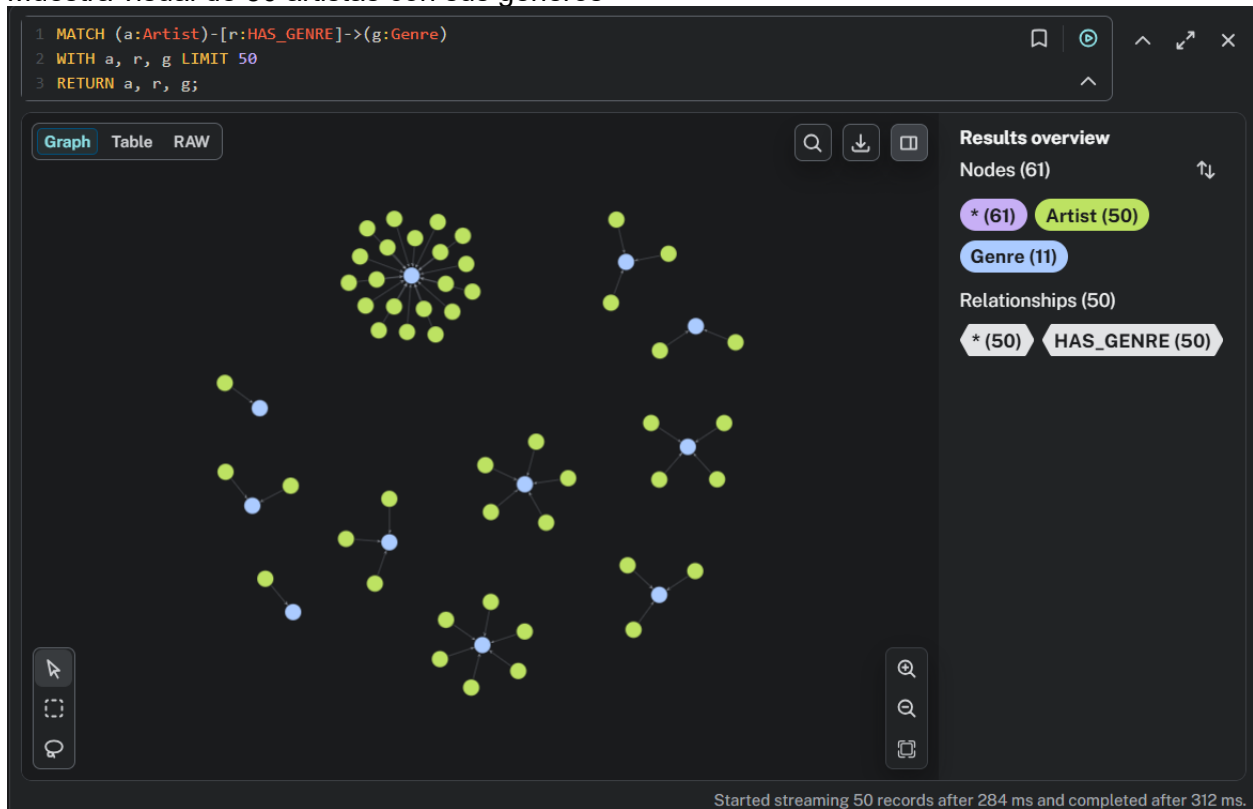
```
spotify$ LOAD CSV WITH HEADERS FROM 'file:///genres.csv' AS row MERGE (:Genre { name: ro
Created 4,853 nodes, set 4,853 properties, added 4,853 labels Completed after 661 ms

spotify$ LOAD CSV WITH HEADERS FROM 'file:///artists.csv' AS row MERGE (a:Artist { spoti
Created 156,316 nodes, set 625,570 properties, added 156,316 labels Completed after 6,923 ms

spotify$ // 0.2 Unicidad de Genre por name CREATE CONSTRAINT genre_name IF NOT EXISTS FO
Added 1 constraint Completed after 58 ms

spotify$ // 0.1 Unicidad de Artist por spotify_id CREATE CONSTRAINT artist_id IF NOT EXI
Added 1 constraint Completed after 207 ms
```

Muestra visual de 50 artistas con sus géneros



3. Análisis Exploratorio Básico (EDA)

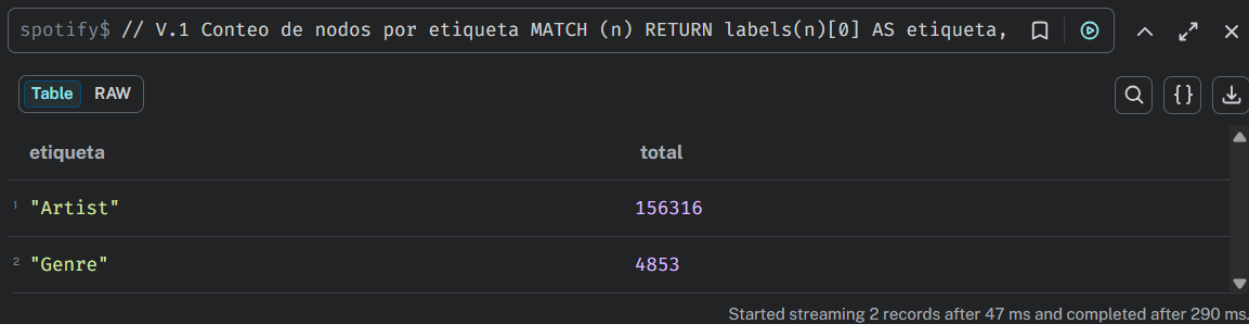
3.1 Conteos generales del grafo

Query: Bloque 1 del archivo proyecto2.cql

3.1.1 Nodos por etiqueta

proyecto2.cql — Query 1.1 (variante unificada)

```
MATCH (n)
RETURN labels(n)[0] AS etiqueta, count(*) AS total
ORDER BY total DESC;
```



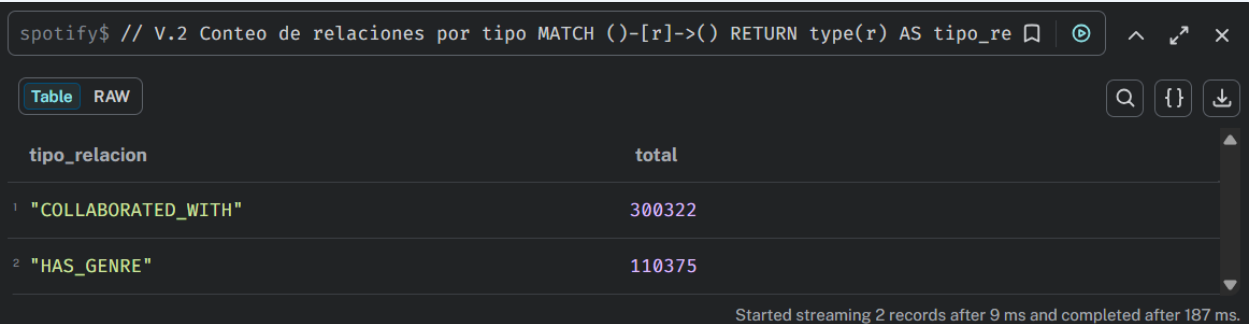
The screenshot shows a query execution interface with a dark theme. At the top, the query is: `spotify$ // V.1 Conteo de nodos por etiqueta MATCH (n) RETURN labels(n)[0] AS etiqueta,`. Below the query, there are tabs for 'Table' and 'RAW'. The 'Table' tab is selected, showing a table with two columns: 'etiqueta' and 'total'. The table contains two rows: 1. 'Artist' with a total of 156316, and 2. 'Genre' with a total of 4853. At the bottom right, a status message reads: 'Started streaming 2 records after 47 ms and completed after 290 ms.'

	etiqueta	total
1	"Artist"	156316
2	"Genre"	4853

3.1.2 Relaciones por tipo

proyecto2.cql — Query 1.2

```
MATCH ()-[r]->()
RETURN type(r) AS tipo_relacion, count(*) AS total
ORDER BY total DESC;
```



The screenshot shows a query execution interface with a dark theme. At the top, the query is: `spotify$ // V.2 Conteo de relaciones por tipo MATCH ()-[r]->() RETURN type(r) AS tipo_re`. Below the query, there are tabs for 'Table' and 'RAW'. The 'Table' tab is selected, showing a table with two columns: 'tipo_relacion' and 'total'. The table contains two rows: 1. 'COLLABORATED_WITH' with a total of 300322, and 2. 'HAS_GENRE' with a total of 110375. At the bottom right, a status message reads: 'Started streaming 2 records after 9 ms and completed after 187 ms.'

	tipo_relacion	total
1	"COLLABORATED_WITH"	300322
2	"HAS_GENRE"	110375

3.1.3 Densidad del subgrafo de colaboraciones

proyecto2.cql — Query 1.3

```
MATCH (a:Artist)
```

```

WITH count(a) AS n
MATCH ()-[r:COLLABORATED_WITH]->()
WITH n, count(r) AS e
RETURN n AS nodos, e AS aristas,
       (2.0*e)/(n*(n-1)) AS densidad;

```

The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (a:Artist) WITH count(a) AS n MATCH ()-[r:COLLABORATED_WITH]->() WITH n, count(r) AS e RETURN n AS nodos, e AS aristas, (2.0*e)/(n*(n-1)) AS densidad;`. The results are displayed in a table with three columns: **nodos**, **aristas**, and **densidad**. The first row shows values 156316, 300322, and 0.000024581764569116915 respectively. A status message at the bottom indicates: "Started streaming 1 record after 380 ms and completed after 619 ms."

nodos	aristas	densidad
156316	300322	0.000024581764569116915

3.1.4 Artistas aislados

proyecto2.cql — Query 1.4 (validación)

```

MATCH (a:Artist) WHERE NOT (a)--()
RETURN count(a) AS artistas_aislados;

```

The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (a:Artist) WHERE NOT (a)--() RETURN count(a) AS artistas_aislados;`. The results are displayed in a table with one column: **artistas_aislados**. The first row shows the value 1717. A status message at the bottom indicates: "Started streaming 1 record after 207 ms and completed after 376 ms."

artistas_aislados
1717

3.2 Estadísticas de atributos

3.2.1 Resumen de popularity y followers

proyecto2.cql — Query 2.1

```
MATCH (a:Artist)
RETURN
    count(a)                AS total,
    min(a.popularity)        AS pop_min,
    max(a.popularity)        AS pop_max,
    round(avg(a.popularity), 2) AS pop_promedio,
    percentileCont(a.popularity, 0.5) AS pop_mediana,
    percentileCont(a.popularity, 0.95) AS pop_p95,
    min(a.followers)         AS fol_min,
    max(a.followers)         AS fol_max,
    round(avg(a.followers), 0) AS fol_promedio,
    percentileCont(a.followers, 0.5) AS fol_mediana,
    percentileCont(a.followers, 0.95) AS fol_p95;
```

total	pop_min	pop_max	pop_prom	pop_med	pop_p95	fol_min	fol_max	fol_prom	fol_med	fol_p95
156316	0	100	21.14	18.0	55.0	0	102156853	86202.0	363.0	213562.25

Started streaming 1 record after 262 ms and completed after 1,006 ms.

3.2.2 Top 10 artistas por popularity

proyecto2.cql — Query 2.2

```
MATCH (a:Artist)
RETURN a.name AS artista, a.popularity AS popularidad,
       a.followers AS seguidores
ORDER BY a.popularity DESC, a.followers DESC
LIMIT 10;
```

spotify\$ MATCH (a:Artist) RETURN a.name AS artista, a.popularity AS popularidad, a.follo

	artista	popularidad	seguidores
1	"Bad Bunny"	100	55669387
2	"Drake"	95	66852536
3	"Taylor Swift"	94	58554324
4	"The Weeknd"	93	49387909
5	"BTS"	91	54532917
6	"Harry Styles"	91	23284667
7	"Kanye West"	91	18170307
8	"Ed Sheeran"	90	102156853
9	"Justin Bieber"	90	65590075
10	"Eminem"	90	59184634

Started streaming 10 records after 261 ms and completed after 437 ms.

3.2.3 Top 10 artistas por followers

proyecto2.cql — Query 2.3

```
MATCH (a:Artist)
RETURN a.name AS artista, a.followers AS seguidores,
       a.popularity AS popularidad
ORDER BY a.followers DESC
LIMIT 10;
```

spotify\$ MATCH (a:Artist) RETURN a.name AS artista, a.followers AS seguidores, a.popular

	artista	seguidores	popularidad
1	"Ed Sheeran"	102156853	90
2	"Ariana Grande"	83045090	88
3	"Billie Eilish"	68407227	88
4	"Drake"	66852536	95
5	"Justin Bieber"	65590075	90
6	"Eminem"	59184634	90
7	"Taylor Swift"	58554324	94
8	"Arijit Singh"	58523986	86
9	"Bad Bunny"	55669387	100
10	"BTS"	54532917	91

Started streaming 10 records after 52 ms and completed after 212 ms.

3.3 Distribución del grado de colaboración

3.3.1 Histograma de grado por bins

proyecto2.cql — Query 3.1

```
MATCH (a:Artist)
WITH a, COUNT { (a)-[:COLLABORATED_WITH]-() } AS grado
WITH
  CASE
    WHEN grado = 0      THEN '0 (aislados)'
    WHEN grado = 1      THEN '1'
    WHEN grado <= 5     THEN '2-5'
    WHEN grado <= 10    THEN '6-10'
    WHEN grado <= 50    THEN '11-50'
    WHEN grado <= 100   THEN '51-100'
    ELSE                '100+'
  END AS bin,
  count(a) AS artistas
RETURN bin, artistas
ORDER BY
  CASE bin
    WHEN '0 (aislados)' THEN 0
    WHEN '1'            THEN 1
    WHEN '2-5'          THEN 2
    WHEN '6-10'         THEN 3
    WHEN '11-50'        THEN 4
    WHEN '51-100'       THEN 5
    ELSE 6
  END;
END;
```

bin	artistas
1 "0 (aislados)"	3038
2 "1"	104207
3 "2-5"	33072
4 "6-10"	6245
5 "11-50"	7893
6 "51-100"	1320
7 "100+"	541

Started streaming 7 records after 354 ms and completed after 548 ms.

3.3.2 Estadísticas del grado

proyecto2.cql — Query 3.2

```

MATCH (a:Artist)
WITH a, COUNT { (a)-[:COLLABORATED_WITH]-() } AS grado
RETURN
  min(grado)           AS grado_min,
  max(grado)           AS grado_max,
  round(avg(grado), 2) AS grado_promedio,
  percentileCont(grado, 0.5) AS grado_mediana,
  percentileCont(grado, 0.95) AS grado_p95,
  percentileCont(grado, 0.99) AS grado_p99;

```

The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (a:Artist) WITH a, COUNT { (a)-[:COLLABORATED_WITH]-() } AS grado RETURN`. The results are displayed in a table with 6 columns: `grado_min`, `grado_max`, `grado_promedio`, `grado_mediana`, `grado_p95`, and `grado_p99`. The first row of data shows values: 0, 1781, 3.84, 1.0, 14.0, and 57.0. A status message at the bottom indicates: "Started streaming 1 record after 239 ms and completed after 543 ms."

	grado_min	grado_max	grado_promedio	grado_mediana	grado_p95	grado_p99
1	0	1781	3.84	1.0	14.0	57.0

3.4 Análisis de géneros

3.4.1 Top 20 géneros con más artistas

proyecto2.cql — Query 5.1

```

MATCH (g:Genre)<-[:HAS_GENRE]-(a:Artist)
WITH g, count(a) AS artistas
ORDER BY artistas DESC
LIMIT 20
RETURN g.name AS genero, artistas;

```

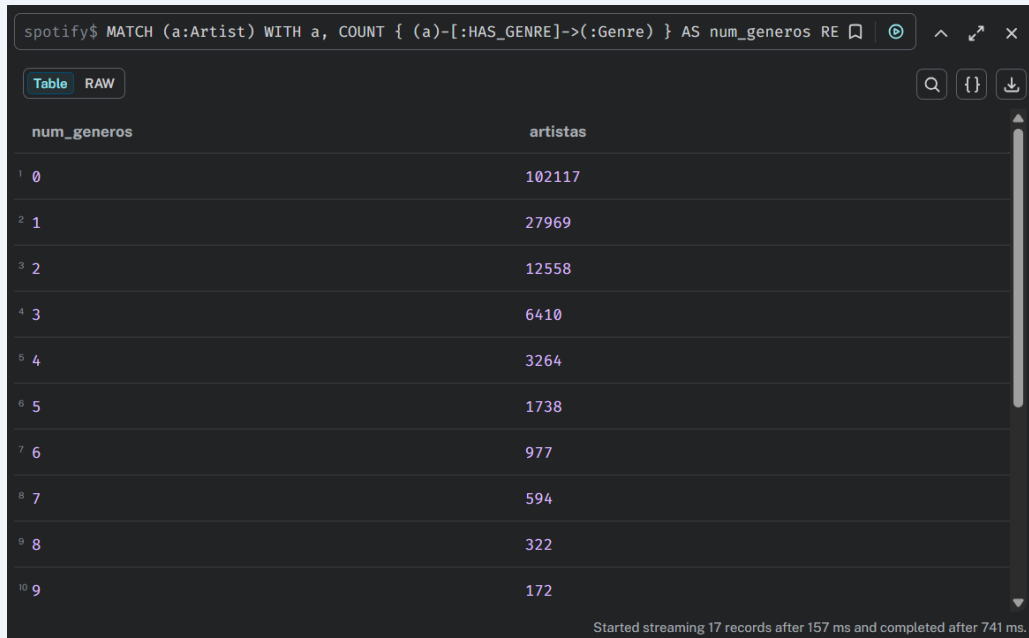
The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (g:Genre)<-[:HAS_GENRE]-(a:Artist) WITH g, count(a) AS artistas ORDER BY`. The results are displayed in a table with 2 columns: `genero` and `artistas`. The first 10 rows of data are shown, listing genres and their corresponding number of artists. A status message at the bottom indicates: "Started streaming 20 records after 275 ms and completed after 484 ms."

	genero	artistas
1	"pop"	544
2	"electro house"	512
3	"dance pop"	505
4	"edm"	452
5	"rap"	448
6	"german hip hop"	434
7	"uk pop"	406
8	"pop rap"	395
9	"french hip hop"	382
10	"hip hop"	381

3.4.2 Distribución de géneros por artista

proyecto2.cql — Query 5.2

```
MATCH (a:Artist)
WITH a, COUNT { (a)-[:HAS_GENRE]->(:Genre) } AS num_generos
RETURN num_generos, count(a) AS artistas
ORDER BY num_generos;
```



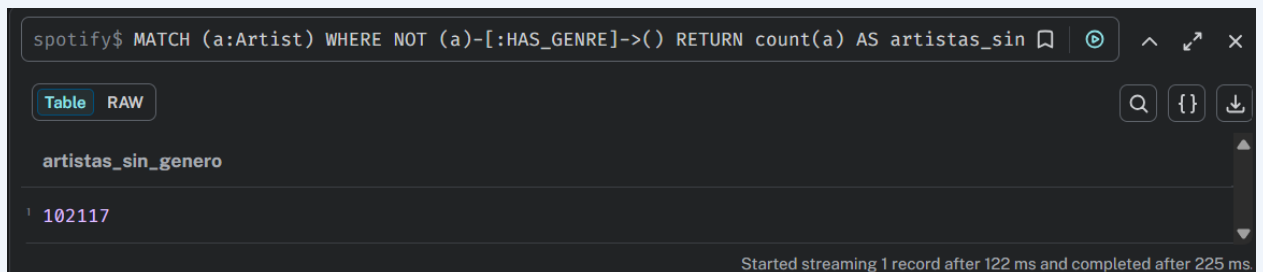
num_generos	artistas
0	102117
1	27969
2	12558
3	6410
4	3264
5	1738
6	977
7	594
8	322
9	172

Started streaming 17 records after 157 ms and completed after 741 ms.

3.4.3 Artistas sin género asignado

proyecto2.cql — Query 5.3

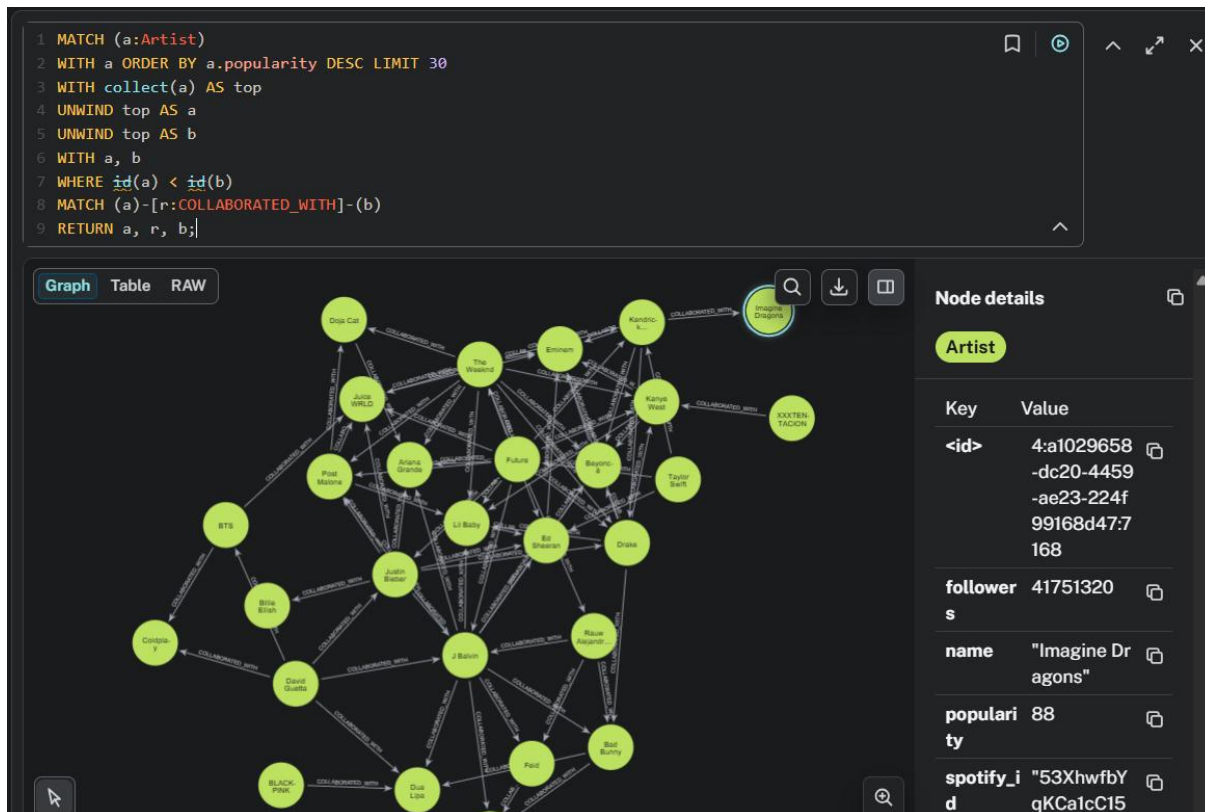
```
MATCH (a:Artist)
WHERE NOT (a)-[:HAS_GENRE]->()
RETURN count(a) AS artistas_sin_genero;
```



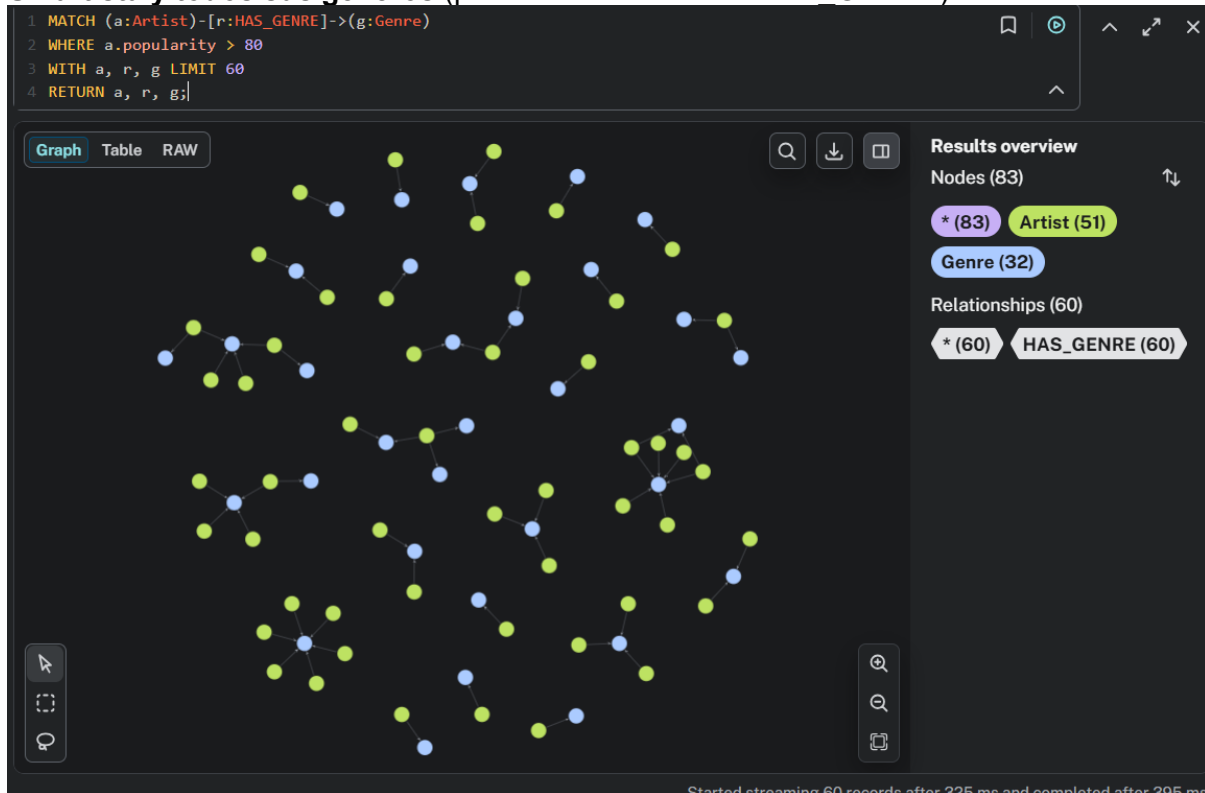
artistas_sin_genero
102117

Started streaming 1 record after 122 ms and completed after 225 ms.

Subgrafo de los 30 artistas más populares y sus colaboraciones entre ellos



Un artista y todos sus géneros (para ilustrar la relación HAS_GENRE)



4. Análisis de Centralidad

4.1 Degree Centrality — Top 20 por colaboraciones

proyecto2.cql — Query 4.1

```
MATCH (a:Artist)
WITH a, COUNT { (a)-[:COLLABORATED_WITH]-() } AS colaboraciones
ORDER BY colaboraciones DESC
LIMIT 20
RETURN
    a.name          AS artista,
    colaboraciones,
    a.popularity    AS popularidad,
    a.followers     AS seguidores;
```

	artista	colaboraciones	popularidad	seguidores
1	"Johann Sebastian Bach"	1781	75	3701532
2	"Traditional"	1370	59	49046
3	"Mc Gw"	858	66	1448741
4	"MC MN"	632	46	32119
5	"Jean Sibelius"	580	56	194098
6	"Armin van Buuren"	513	74	4094260
7	"Gucci Mane"	509	77	4454550
8	"Steve Aoki"	498	74	3689746
9	"Snoop Dogg"	495	84	8686870
10	"Diplo"	494	79	2498486

Started streaming 20 records after 151 ms and completed after 289 ms.

4.2 Betweenness Centrality (GDS)

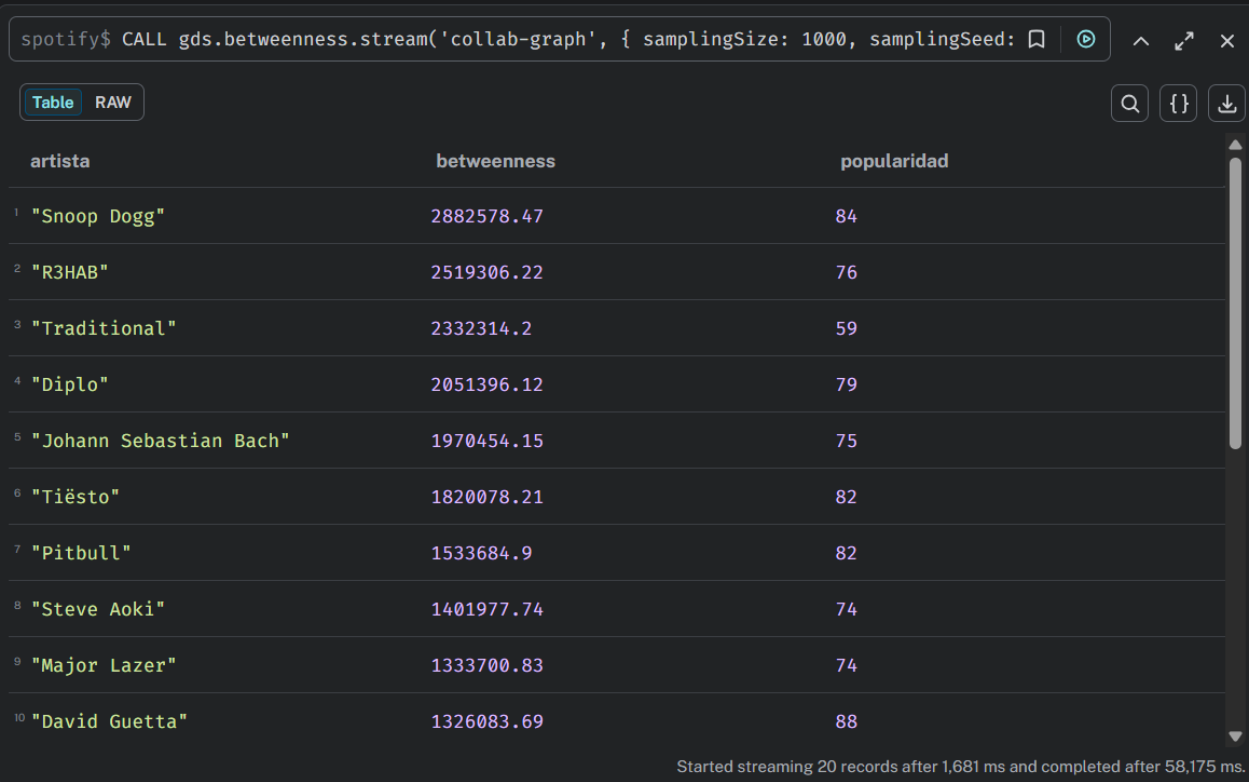
Requiere Graph Data Science Library. Proyectar el grafo y ejecutar el algoritmo.

Proyección del grafo nativo

```
CALL gds.graph.project(
    'collab-graph',
    'Artist',
    { COLLABORATED_WITH: { orientation: 'UNDIRECTED' } }
);
```

Betweenness Centrality — Top 20

```
CALL gds.betweenness.stream('collab-graph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS a, score
ORDER BY score DESC
LIMIT 20
RETURN a.name AS artista, round(score, 2) AS betweenness,
       a.popularity AS popularidad;
```



	artista	betweenness	popularidad
1	"Snoop Dogg"	2882578.47	84
2	"R3HAB"	2519306.22	76
3	"Traditional"	2332314.2	59
4	"Diplo"	2051396.12	79
5	"Johann Sebastian Bach"	1970454.15	75
6	"Tiësto"	1820078.21	82
7	"Pitbull"	1533684.9	82
8	"Steve Aoki"	1401977.74	74
9	"Major Lazer"	1333700.83	74
10	"David Guetta"	1326083.69	88

Started streaming 20 records after 1,681 ms and completed after 58,175 ms.

4.3 PageRank (GDS)

PageRank — Top 20

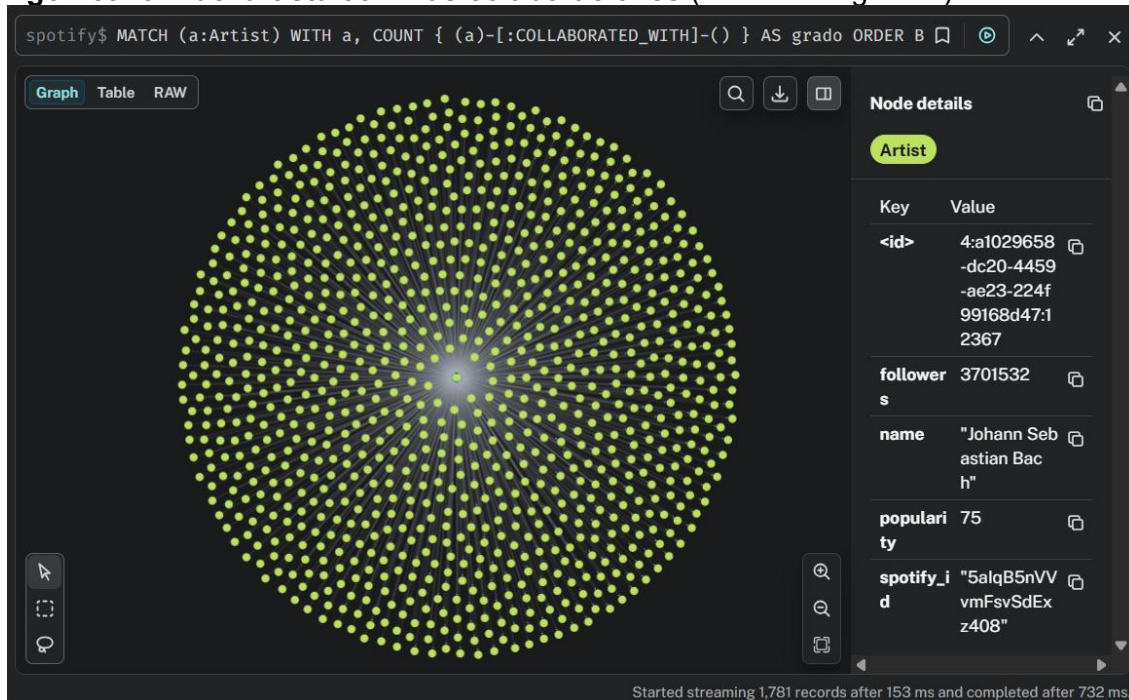
```
CALL gds.pageRank.stream('collab-graph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS a, score
ORDER BY score DESC
LIMIT 20
RETURN a.name AS artista, round(score, 4) AS pagerank,
       a.popularity AS popularidad;
```

```
spotify$ CALL gds.pageRank.stream('collab-graph') YIELD nodeId, score WITH gds.util.asNo
```

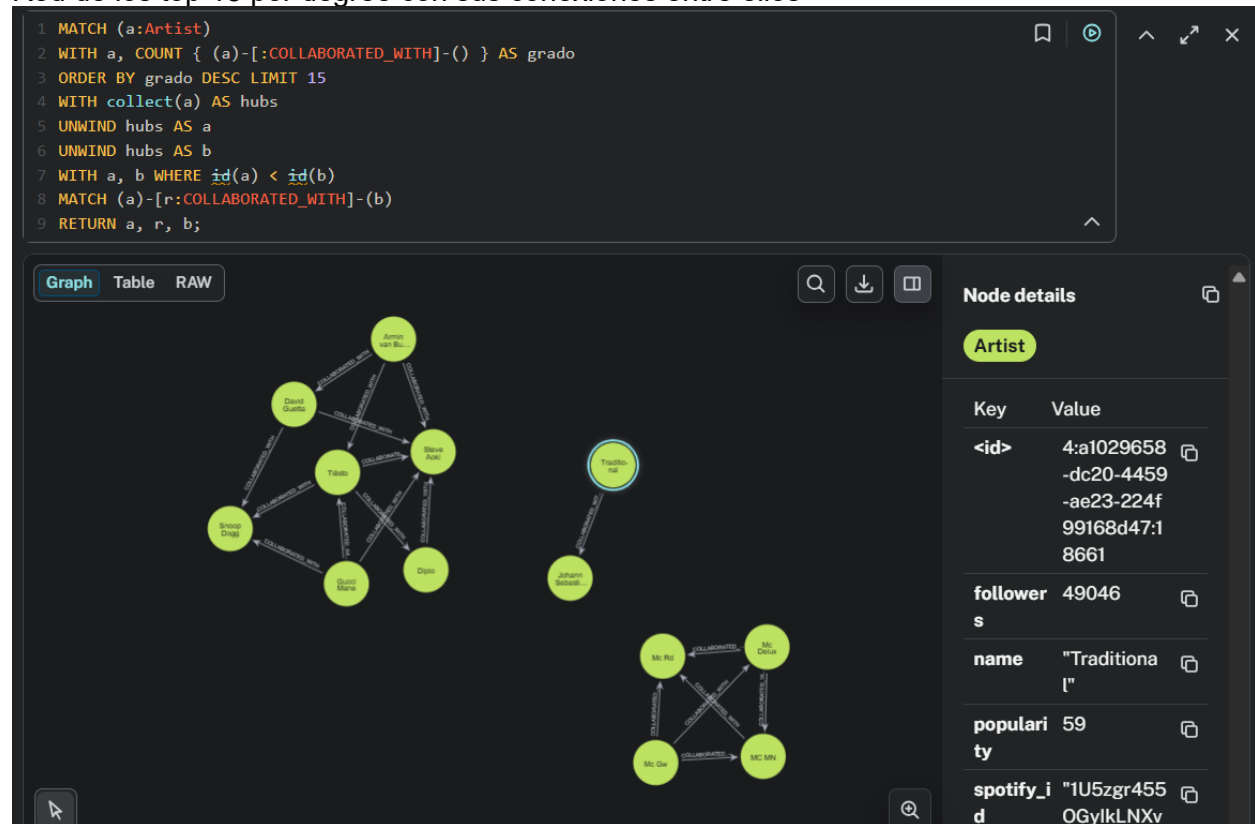
	artista	pagerank	popularidad
1	"Johann Sebastian Bach"	725.6797	75
2	"Traditional"	527.3687	59
3	"Jean Sibelius"	215.8821	56
4	"Mc Gw"	167.7766	66
5	"הכוכב הבא"	144.6709	48
6	"MC MN"	135.3349	46
7	"John Williams"	134.7102	70
8	"A.R. Rahman"	108.41	79
9	"Armin van Buuren"	103.5784	74
10	"Andrea Bocelli"	99.9827	67

Started streaming 20 records after 368 ms and completed after 2.500 ms

Ego-network del artista con más colaboraciones (el hub más grande)



Red de los top 15 por degree con sus conexiones entre ellos



5. Análisis de Comunidades

5.1 Louvain (GDS)

Detección de comunidades con Louvain

```
CALL gds.louvain.stream('collab-graph')
YIELD nodeId, communityId
WITH gds.util.asNode(nodeId) AS a, communityId
RETURN communityId,
       count(a) AS miembros
ORDER BY miembros DESC
LIMIT 20;
```

	communityId	miembros
1	100509	23769
2	101843	14016
3	105607	9644
4	105704	8573
5	102372	7057
6	103165	6449
7	102109	4557
8	109485	4440
9	107714	3975
10	107757	3599

Started streaming 20 records after 273 ms and completed after 14,145 ms.

5.2 Top artistas por comunidad

Artistas representativos de las 5 comunidades más grandes

```
CALL gds.louvain.stream('collab-graph')
YIELD nodeId, communityId
WITH gds.util.asNode(nodeId) AS a, communityId
ORDER BY a.popularity DESC
WITH communityId, collect(a.name)[..5] AS top_artistas, count(a) AS miembros
ORDER BY miembros DESC
LIMIT 5
RETURN communityId, miembros, top_artistas;
```

spotify\$ CALL gds.louvain.stream('collab-graph') YIELD nodeId, communityId WITH gds.util

Table RAW

🔍

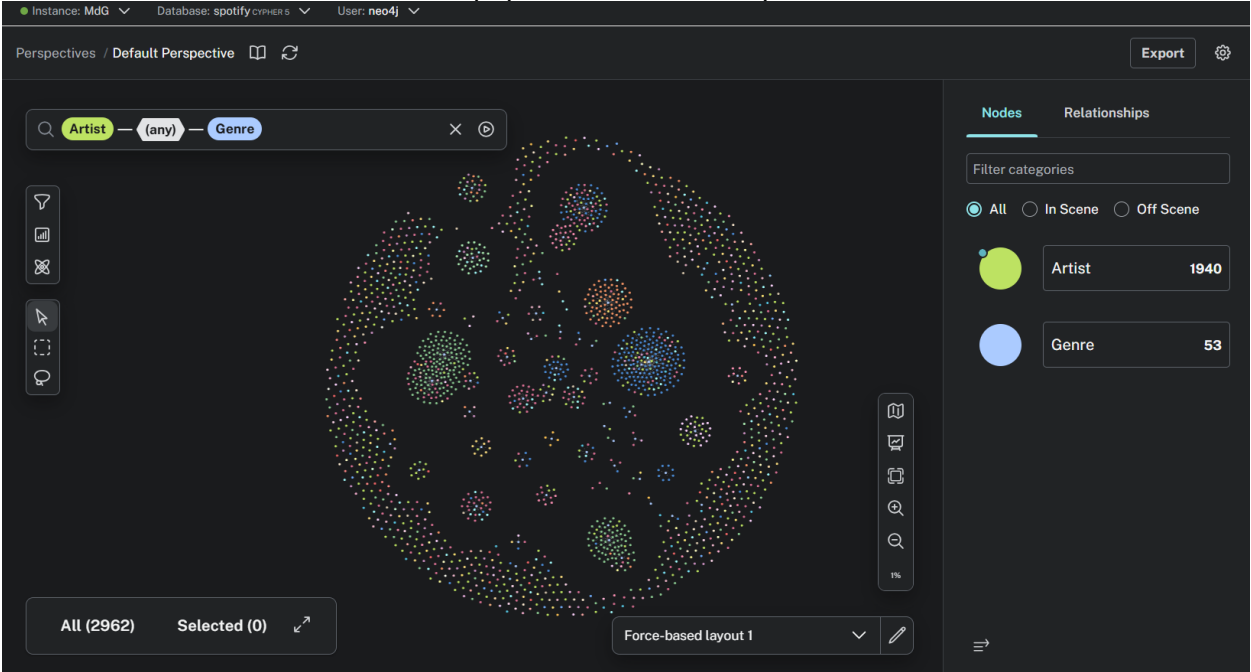
{ }

⬇

	communityId	miembros	top_artistas
1	100509	25174	["Taylor Swift", "BTS", "Ed Sheeran", "Dua Lipa", "Imagine Dragons"]
2	107282	14461	["Bad Bunny", "Juice WRLD", "Rauw Alejandro", "Post Malone", "Feid"]
3	106673	9557	["Drake", "The Weeknd", "Kanye West", "Eminem", "Justin Bieber"]
4	105704	8746	["Marília Mendonça", "Henrique & Juliano", "Jorge & Mateus", "Anitta", "Gusttavo Lima"]
5	116517	7151	["Olivia Rodrigo", "The Beatles", "Disney", "Elvis Presley", "Ludovico Einaudi"]

Started streaming 5 records after 517 ms and completed after 12,108 ms.

Louvain visual: los 200 artistas más populares coloreados por comunidad



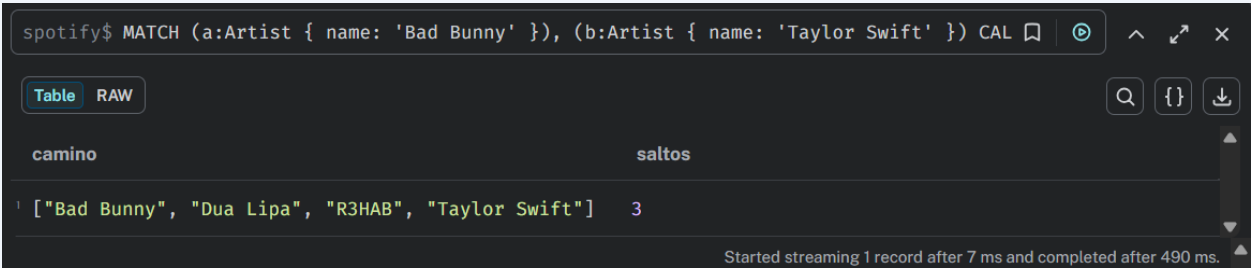
6. Búsqueda de Caminos

6.1 Camino más corto entre dos artistas

Sustituye los nombres de los artistas por los que sean de interés para tu análisis.

Shortest Path — ejemplo

```
MATCH (a:Artist { name: 'Bad Bunny' }),
      (b:Artist { name: 'Taylor Swift' })
CALL gds.shortestPath.dijkstra.stream('collab-graph', {
  sourceNode: id(a),
  targetNode: id(b)
})
YIELD index, sourceNode, targetNode, totalCost, nodeIds, path
RETURN
  [nodeId IN nodeIds | gds.util.asNode(nodeId).name] AS camino,
  length(path) AS saltos;
```

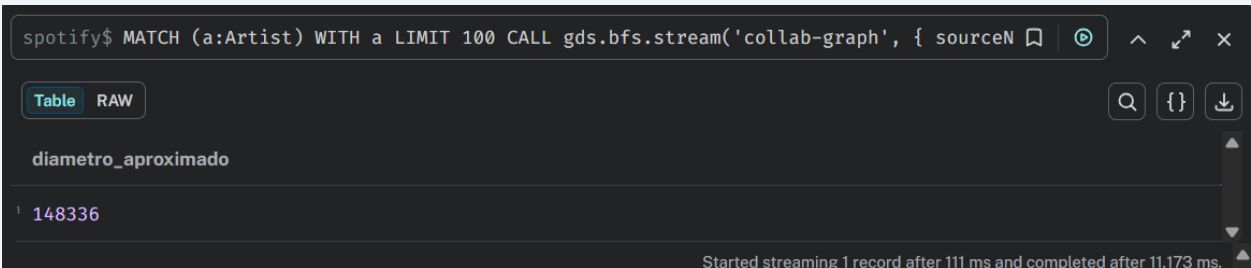


The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (a:Artist { name: 'Bad Bunny' }), (b:Artist { name: 'Taylor Swift' }) CALL gds.shortestPath.dijkstra.stream('collab-graph', { sourceNode: id(a), targetNode: id(b) }) YIELD index, sourceNode, targetNode, totalCost, nodeIds, path RETURN [nodeId IN nodeIds | gds.util.asNode(nodeId).name] AS camino, length(path) AS saltos;` The interface has tabs for 'Table' and 'RAW'. The 'Table' view shows a single row with two columns: 'camino' and 'saltos'. The 'camino' column contains the list `["Bad Bunny", "Dua Lipa", "R3HAB", "Taylor Swift"]` and the 'saltos' column contains the value `3`. At the bottom, a status message reads: "Started streaming 1 record after 7 ms and completed after 490 ms."

6.2 Diámetro de la red (camino más largo entre artistas alcanzables)

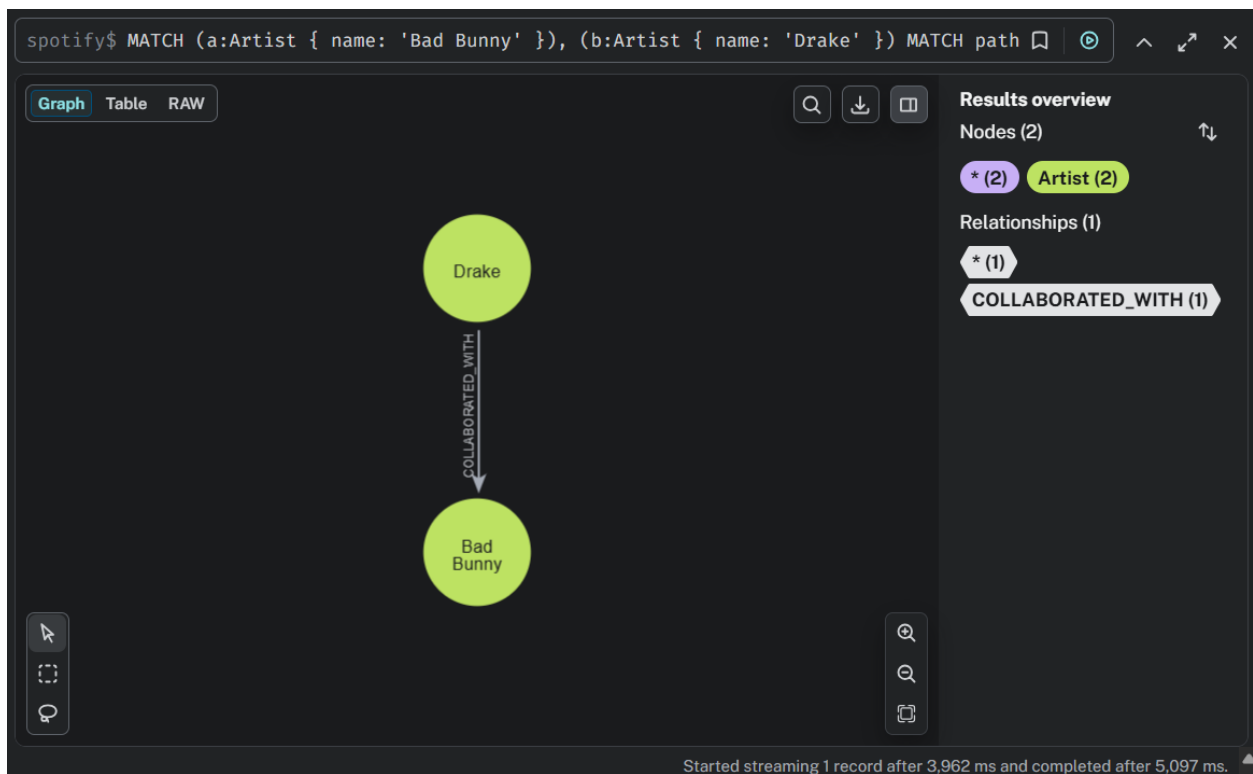
Diámetro aproximado con BFS (muestra de 100 artistas)

```
MATCH (a:Artist)
WITH a LIMIT 100
CALL gds.bfs.stream('collab-graph', { sourceNode: id(a) })
YIELD path
RETURN max(length(path)) AS diametro_aproximado;
```

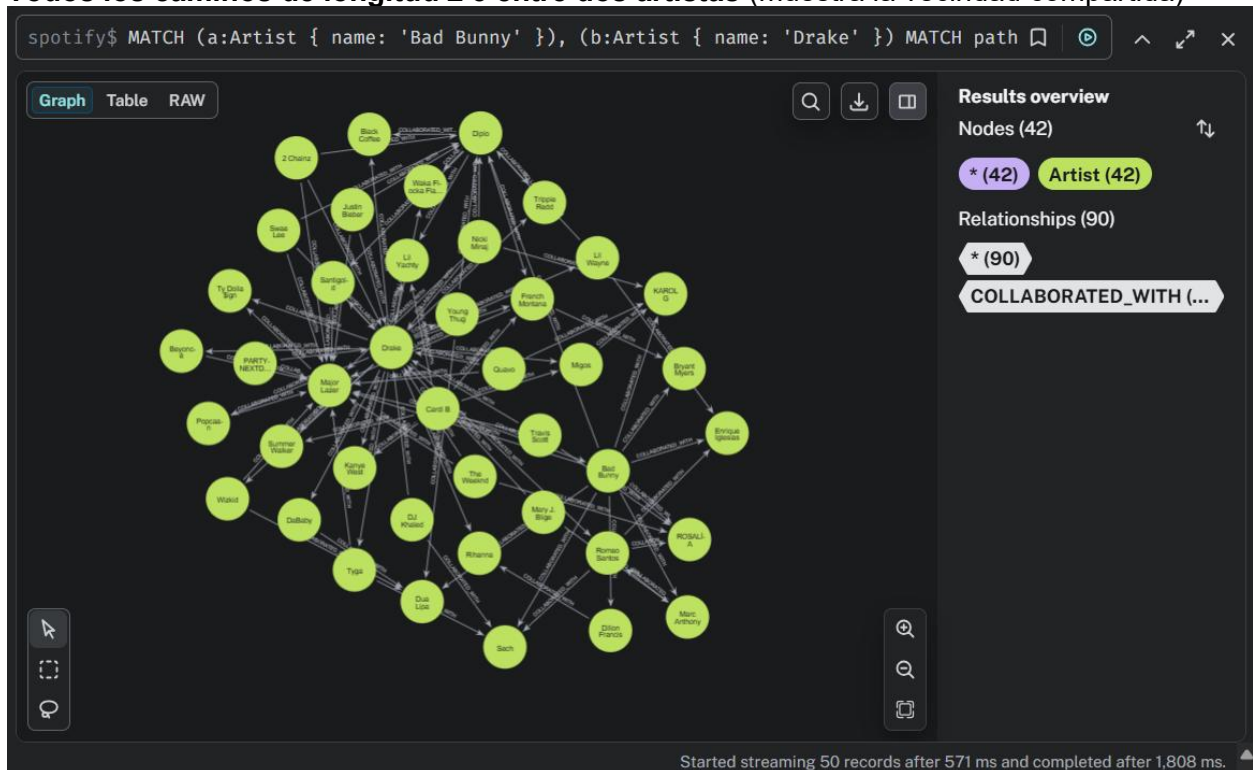


The screenshot shows a Neo4j Cypher query interface. The query is: `spotify$ MATCH (a:Artist) WITH a LIMIT 100 CALL gds.bfs.stream('collab-graph', { sourceNode: id(a) }) YIELD path RETURN max(length(path)) AS diametro_aproximado;` The interface has tabs for 'Table' and 'RAW'. The 'Table' view shows a single row with one column: 'diametro_aproximado'. The column contains the value `148336`. At the bottom, a status message reads: "Started streaming 1 record after 111 ms and completed after 11,173 ms."

Camino más corto visual entre dos artistas

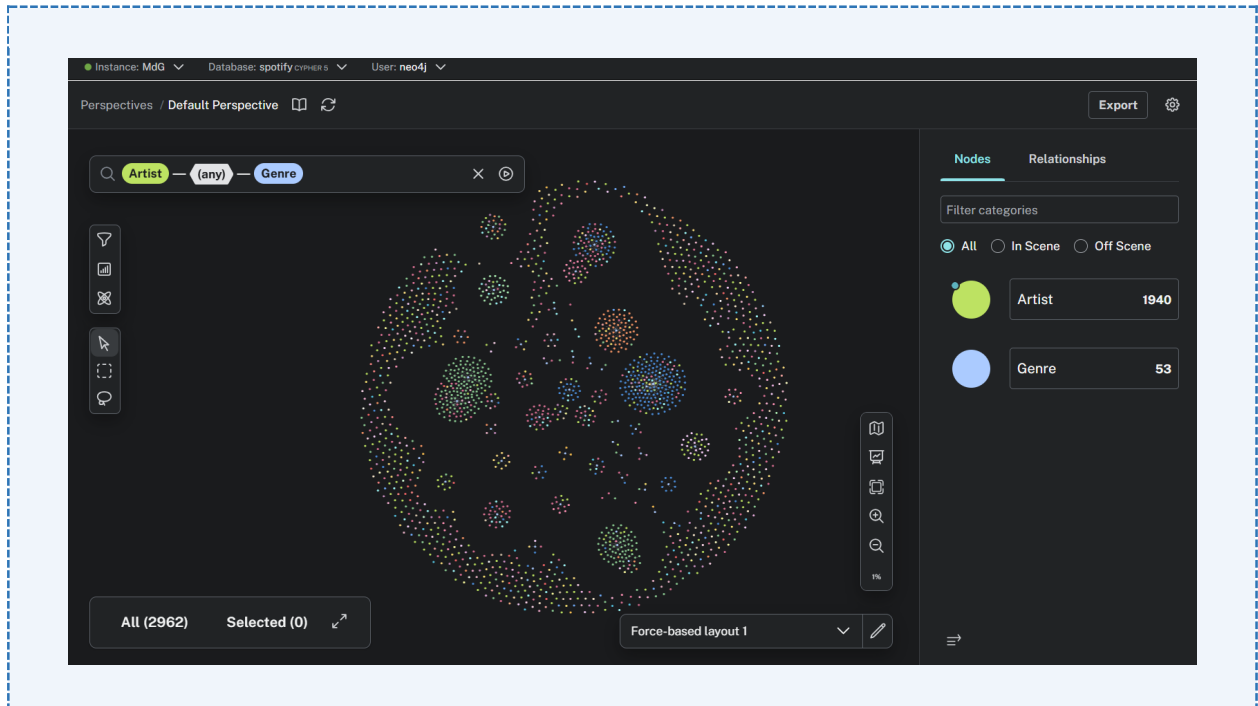


Todos los caminos de longitud ≤ 3 entre dos artistas (muestra la vecindad compartida)

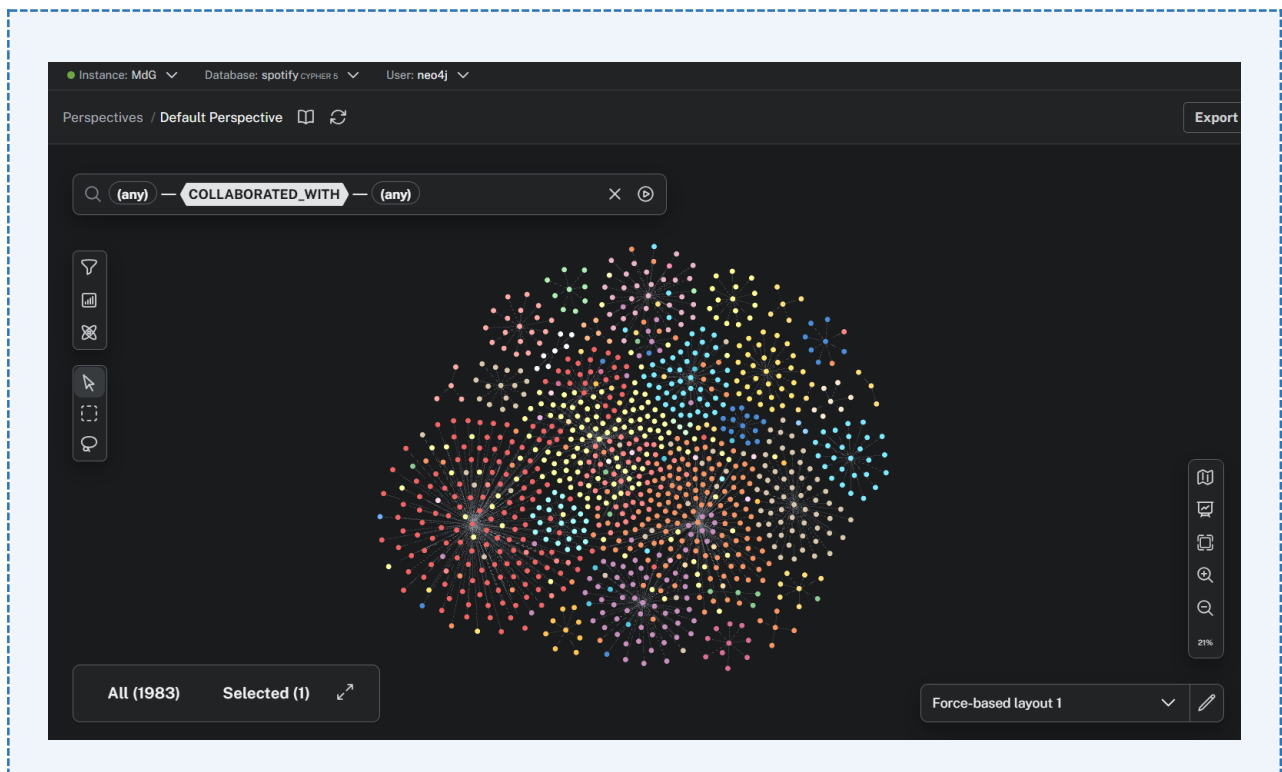


7. Visualización con Neo4j Explore / Bloom

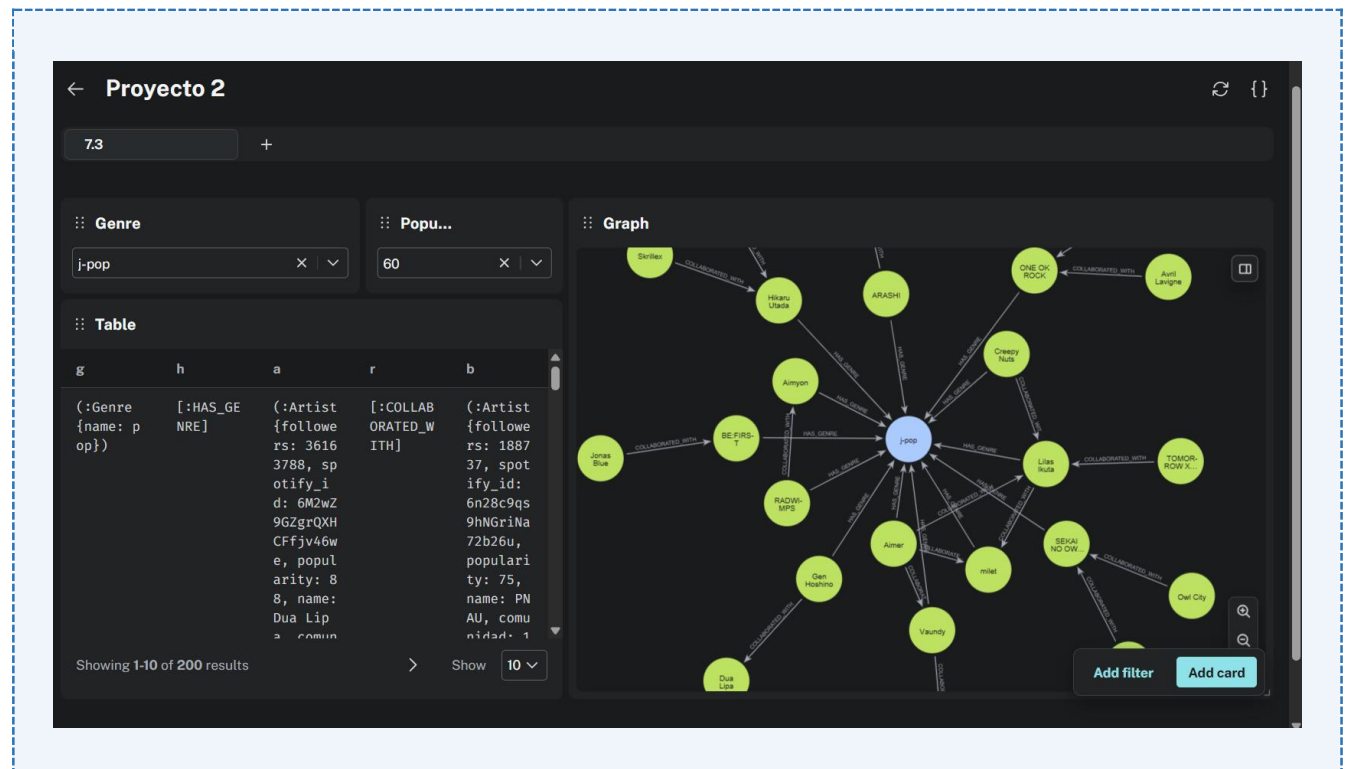
7.1 Vista general del grafo



7.2 Comunidades con color



7.3 Dashboard (NeoDash)



8. Conclusiones

8.1 Hallazgos principales

(Completar con los resultados obtenidos)

C1. Sobre la estructura de la red:

La red de colaboraciones de Spotify está compuesta por 156,316 artistas y 300,322 relaciones COLLABORATED_WITH, complementadas por 4,853 géneros conectados mediante 110,375 relaciones HAS_GENRE. La densidad del grafo de colaboraciones es extremadamente baja (0.0000246), lo que confirma que se trata de una red dispersa característica de redes del mundo real. La distribución del grado es fuertemente asimétrica (ley de potencias): la inmensa mayoría de los artistas tiene solo 1 colaboración (104,207 artistas), mientras que un pequeño núcleo de 541 artistas supera las 100 colaboraciones y el máximo alcanza 1,781. El grado promedio es 3.84 y la mediana es 1, lo que indica una red dominada por artistas periféricos con pocos vínculos.

C2. Sobre los artistas más centrales:

Los tres enfoques de centralidad revelan perfiles distintos. Por **degree**, Johann Sebastian Bach (1,781 colaboraciones) y Traditional (1,370) encabezan la lista, revelando que categorías de música clásica y folclórica actúan como hubs de alta conectividad a pesar de tener popularidad moderada en Spotify. Por **betweenness** (muestreo de 1,000 nodos), Snoop Dogg lidera con 2,882,578 puntos, seguido de R3HAB (2,519,306) y Traditional (2,332,314), lo que indica que estos artistas son puentes críticos entre distintas comunidades musicales. Por **PageRank**, Johann Sebastian Bach (725.68) y Traditional (527.37) vuelven a dominar, confirmando que sus conexiones provienen de nodos también muy conectados. Notablemente, los artistas más populares de Spotify (Bad Bunny, Drake, Taylor Swift) no coinciden con los más centrales en la red, lo que sugiere que popularidad y centralidad estructural son métricas independientes en este grafo.

C3. Sobre las comunidades detectadas:

El algoritmo de Louvain detectó comunidades claramente identificables por género y región geográfica. La comunidad más grande (100509, con 25,174 miembros) agrupa artistas de pop anglosajón mainstream: Taylor Swift, BTS, Ed Sheeran, Dua Lipa e Imagine Dragons. La segunda (107282, 14,461 miembros) concentra el urbano latino: Bad Bunny, Juice WRLD, Rauw Alejandro, Post Malone y Feid. La tercera (106673, 9,557 miembros) corresponde al hip-hop y R&B norteamericano: Drake, The Weeknd, Kanye West, Eminem y Justin Bieber. La cuarta (105704, 8,746 miembros) es prácticamente exclusiva del sertanejo brasileño: Marília Mendonça, Henrique & Juliano, Jorge & Mateus, Anitta y Gustavo Lima. Esta segmentación confirma que las colaboraciones musicales siguen fronteras de idioma, género y mercado regional con alta consistencia.

C4. Sobre la relación entre género y colaboración:

El género más representado es pop (544 artistas), seguido de electro house (512), dance pop (505) y EDM (452), lo que refleja el predominio de géneros electrónicos y pop en la plataforma. Sin embargo, el 65.3% de los artistas (102,117 de 156,316) no tienen ningún género asignado, lo que es una limitación estructural del dataset. Entre los artistas con género asignado, la mayoría tiene entre 1 y 3 géneros. El análisis visual en NeoDash (filtro por género j-pop con popularidad > 60) muestra cómo un nodo de género actúa como eje gravitacional de su

comunidad, conectando artistas que colaboran entre sí casi exclusivamente dentro de su ecosistema, como se observa con el clúster de j-pop (RADWIMPS, Aimer, Gen Hoshino, Aimer, etc.).

8.2 Limitaciones

- **Cobertura de géneros incompleta:** el 65.3% de los artistas no tiene género asignado en el dataset, lo que limita el análisis de la relación entre género y colaboración al subconjunto restante.
- **Grafo no temporal:** el dataset no incluye fechas de colaboración, por lo que no es posible analizar la evolución de la red en el tiempo ni identificar tendencias emergentes.
- **Definición de colaboración limitada al "feature":** la red captura únicamente colaboraciones formales (apariciones acreditadas en tracks), excluyendo influencias, co-escritura o producciones no acreditadas.
- **Betweenness aproximado:** dada la escala de 156k nodos, el cálculo exacto de betweenness es computacionalmente inviable; se usó muestreo de 1,000 nodos fuente, lo que introduce un margen de error en el ranking.
- **Artistas aislados:** 1,717 artistas no tienen ninguna relación en el grafo, posiblemente por errores en los IDs del dataset original o artistas sin colaboraciones registradas.

8.3 Trabajo futuro

- **Análisis temporal:** incorporar fechas de lanzamiento de los tracks para construir una red dinámica y estudiar cómo evolucionan las comunidades y los hubs a lo largo del tiempo.
- **Enriquecimiento del grafo:** integrar atributos de audio de la API de Spotify (tempo, danceability, valence) como propiedades de arista para analizar si las colaboraciones ocurren entre artistas con sonidos similares o complementarios.
- **Análisis de similitud de géneros:** usar el bipartito artista-género para construir una proyección género-género y estudiar qué géneros tienden a coexistir en los mismos artistas, generando un mapa de proximidad musical.
- **Predicción de enlaces:** aplicar algoritmos de link prediction (Common Neighbors, Adamic-Adar) para predecir qué artistas son más probables de colaborar en el futuro.
- **Comparación entre mercados:** segmentar el grafo por región geográfica inferida a partir del género (reggaeton → latinoamérica, k-pop → corea) y comparar métricas estructurales entre submercados.

9. Referencias

Ortega-Vázquez, V., et al. (2024). A Methodology for Knowledge Discovery in Labeled and Heterogeneous Graphs. *Applied Sciences*, 14(2), 838. <https://doi.org/10.3390/app14020838>

Freyberg, J. (2022). Spotify Artist Feature Collaboration Network. Kaggle. <https://www.kaggle.com/datasets/jfreyberg/spotify-artist-feature-collaboration-network>

Neo4j. (2024). Graph Data Science Library. <https://neo4j.com/docs/graph-data-science/current/algorithms/>

Neo4j. (2024). Cypher Cheat Sheet. <https://neo4j.com/docs/cypher-cheat-sheet/5/neo4j-enterprise>

Neo4j. (2024). NeoDash. <https://neo4j.com/labs/neodash/>